Tomasulo Algorithm Implementation: Multiple Issue

Daniel Prol - UH ID: 2446940

1. Summary

I started from the Python Tomasulo simulator (single issue, ROB, RAT, load/store queue with memory disambiguation) and added in-order issue of 1 to 4 instructions per cycle. The issue width is read from the input file (`Issue width = n`, default 1) or given on the command line.

What works:

- Issue width 1 to 4, in program order. With issue width n, up to n results are broadcast on the CDB and up to n instructions commit per cycle, as in the multi-issue example of the class slides.
- The hardware configuration, initial register and memory values and the program are read from an input file in the format of the assignment.
- The output has the timing table and the final register and non-zero memory values.
- The simulator gives the same timing table as the example of the class slides (OOO example) for issue width 1 and 2.
- All 6 test cases give the same final registers and memory as a sequential reference model for every issue width (24 runs).

What does not work / limitations:

- There is no branch prediction. The starting code stalls fetch after a branch until the branch is resolved in EX, and I kept that. With multiple issue, no instruction after a branch is issued in the same cycle. The 1-bit predictor and the BTB of the assignment text are not implemented, so there is no speculation and no squashing.
- Only 1 functional unit per type is supported (a different `# of FUs` gives a warning and 1 is used).
- The branch offset is in bytes, as in the starting code and its test case (`Bne R1 R0 -12` goes 3 instructions back), not in words as in the assignment text.

2. What I added to the code

2.1 Multiple issue (`issue.py`, `main.py`)

The original `issue` function is now `issue_one` and returns whether the instruction was issued. The new `issue` calls it up to `issue_width` times in the same cycle:

```
def issue(cycle, PC, instructions, ROB, size_ROB, rs_int_adder, rs_fp_adder,
          rs_fp_multi, ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp, issue_width):
    for _ in range(issue_width):
        # nothing left to fetch, or fetch is stalled by an unresolved branch
        if (PC.PC>=len(instructions))|(PC.valid==0):
            break
        # in-order issue: stop at the first instruction that has no ROB/RS space
        if not issue_one(cycle, PC, instructions, ROB, size_ROB, rs_int_adder,
                         rs_fp_adder, rs_fp_multi, ld_sd_queue, size_ld_sd_queue,
                         rat_int, rat_fp):
            break
```

Three things keep the issue in order:

- The loop stops at the first instruction that cannot be issued (ROB, reservation station or load/store queue full), so a later instruction never passes it.
- The RAT is updated after each instruction, so an instruction that depends on another one issued in the same cycle gets the ROB tag of that instruction.
- The loop stops after a branch, because the next PC is not known until the branch is resolved.

2.2 Write back and commit (`wb.py`, `commit.py`)

With issue width n, the CDB broadcasts up to n results per cycle (`wb.py`) and up to n instructions leave the ROB per cycle (`commit.py`). These are the assumptions of the multi-issue example of the class slides, where two instructions issue, two results are broadcast and two instructions commit per cycle. With issue width 1 there is one broadcast per cycle, as in the original code.

`commit` was rewritten. The instructions leave the ROB in program order and the loop stops at the first one that is not finished. Branches now get a commit cycle and take one commit slot (in the original code they were removed from the ROB without a commit cycle).

2.3 Input and output (`init.py`, `main.py`)

- `read_input` parses the input file: reservation stations and cycles of each unit, ROB entries, issue width, initial registers and memory, and the instructions. In the original code these values were written in `main.py`.
- Instructions are case insensitive and operands can be separated with commas (`ld F4, 8(R1)`). Register numbers with two digits work in `Ld`/`Sd`.
- `Beq` was added (the original code had only `Bne`).
- R0 is hardwired to 0.
- After the timing table the simulator prints the total cycles, all registers and the non-zero memory values.

2.4 Problems of the original code that I fixed

Some of these were hidden with single issue and appeared with multiple issue; the others I found with the reference model of section 3.

File	Problem	Fix
`wb.py`	A broadcast always wrote its value in the RAT, also when a later instruction had already renamed the register. Later instructions then read an old value.	The RAT entry is updated only if it still has the tag of the broadcast.
`exe.py`	An instruction issued in a cycle was moved to the functional unit in that same cycle, so with 2 issues per cycle two instructions used the single integer adder (or address adder) in the same cycle.	Only instructions issued in a previous cycle are selected, the oldest first. The integer adder takes a new instruction only when it is free (unpipelined).
`exe.py`	When an instruction entered a pipelined unit, all the instructions already in the unit advanced a second time in that cycle and finished early.	Only the new instruction advances.
`exe.py`	The address calculation never finished with more than 1 cycle in EX.	The cycle counter increases every cycle.

File	Problem	Fix
`mem.py`	`check_if_sd_committable` was never called (the `if` tested the function name), so a store could write memory before older instructions committed.	A store goes to memory only when it is at the head of the ROB.
`mem.py`	The forwarding loop reused its index variable, and a load could be forwarded in its EX cycle.	Separate variable; forwarding only after the EX cycle.
`mem.py`	A load could not go to memory while a store with data was at the head of the queue, also with different addresses.	The check was removed; `check_all_previous_Sd_no_match` already does the disambiguation.
`main.py`	The simulation ended when the ROB was empty, also if a branch had just been resolved and instructions were left, or if the last store was still writing.	The loop also continues while there are instructions to fetch or the memory unit is busy.

With issue width 1 the timing of the provided test case is the same as the original code, except that the three branches now have a commit cycle (35, 38, 41) and the instructions after them commit one cycle later for each branch (the last commit is in cycle 42 instead of 39).

3. Test cases

The test cases are in `tests/`. `code/run_tests.py` runs each one with issue width 1 to 4 and checks:

- the committed instructions, the final registers and the final memory against a sequential reference model (one instruction at a time, no pipeline);
- at most `issue width` instructions issued, broadcast and committed per cycle, with issue and commit in program order;
- every instruction goes through ISSUE, EX, MEM, WB, COMMIT in increasing cycles, with the configured number of cycles in EX and MEM;
- one instruction at a time in the integer adder, the address adder and the memory port, and stores write memory at commit;
- for `test6_class_example`, the complete timing table against the tables of the class slides (issue width 1 and 2).

Test	What it tests
`test1_loop`	The provided test case (`code/test_case.txt`): loop with a `Bne`.
`test2_independent`	12 instructions with few dependencies and no branches.
`test3_forwarding`	The sample configuration of the assignment text, with comma syntax. Store to load forwarding, loads that pass stores with a different address, load/store queue of 3 entries that fills.
`test4_small_structures`	ROB of 4 entries and 1 reservation station per unit, 2 cycles in the integer adder, a register renamed twice (`F7`), writes to R0. Structural hazards stop the issue in the middle of a group.

Test	What it tests
`test5_branches`	Lower case input, a loop with load, multiply, add, store and load of the same address, `Bne` taken 5 times, `Beq` not taken and `Beq` taken (skips one instruction).
`test6_class_example`	The example of the class slides (OOO example): 2 loads, 2 multiplies, 1 FP add and 1 integer add, with the latencies of the slides.

3.1 Total cycles for each issue width

Test	Width 1	Width 2	Width 3	Width 4	Result
`test1_loop`	42	36	35	35	pass
`test2_independent`	21	17	17	16	pass
`test3_forwarding`	50	48	47	47	pass
`test4_small_structures`	48	46	46	46	pass
`test5_branches`	69	72	70	69	pass
`test6_class_example`	40	39	38	38	pass

The final registers and memory are the same for the 4 widths in every test and equal to the reference model. The complete output of the 24 runs is in `results/`.

`test2_independent` has the largest gain (21 to 16 cycles) because its instructions have few dependencies. In the other tests the gain is small because the rest of the machine is the same: one unit of each type, one memory port, and fetch stalled after every branch. The instructions enter the ROB earlier, but then they wait for a long multiply, for the memory port or for a reservation station.

In `test5_branches` width 2 and 3 are slower than width 1 and width 4 is equal. This is not an error in the results (they are equal to the reference model); it is contention. With a wider issue the stores of one iteration enter the 4-entry load/store queue earlier and stay there until they commit, so the queue is full and the issue stalls (with width 2, `Ld F4 0(R3)` of the second iteration is issued in cycle 19, 7 cycles after the store before it). The first load of the next iteration also waits for the memory port behind two stores that are committing. With a load/store queue of 6 entries the same program takes 65, 60, 59 and 59 cycles.

3.2 `test2_independent`

Configuration: integer adder 4 RS, 1 cycle; FP adder 4 RS, 3 cycles; FP multiplier 4 RS, 5 cycles; load/store 4 entries, 1 cycle EX, 2 cycles MEM; ROB 64. R1=8, R2=16, R3=5, R4=7, F1=1.5, F2=2.5, F3=4.0, Mem[8]=10.0, Mem[16]=20.0.

Issue width 1:

	ISSUE	EXE	MEM	WB	COMMIT
`Ld F4 0(R1)`	[1]	[2, 2]	[3, 4]	[5]	[6]
`Ld F5 0(R2)`	[2]	[3, 3]	[5, 6]	[9]	[10]
`Add.d F6 F1 F2`	[3]	[4, 6]	[]	[7]	[11]
`Mult.d F7 F2 F3`	[4]	[5, 9]	[]	[11]	[12]
`Add R5 R3 R4`	[5]	[6, 6]	[]	[8]	[13]
`Sub R6 R4 R3`	[6]	[7, 7]	[]	[10]	[14]
`Sub.d F8 F3 F1`	[7]	[8, 10]	[]	[12]	[15]
`Mult.d F9 F1 F1`	[8]	[9, 13]	[]	[15]	[16]
`Addi R7 R3 100`	[9]	[10, 10]	[]	[13]	[17]
`Add.d F10 F4 F5`	[10]	[11, 13]	[]	[14]	[18]
`Mult.d F11 F6 F7`	[11]	[12, 16]	[]	[17]	[19]
`Sd F10 8(R2)`	[12]	[13, 13]	[20, 21]	[]	[20]

Issue width 4:

	ISSUE	EXE	MEM	WB	COMMIT
Ld F4 0(R1)	[1]	[2, 2]	[3, 4]	[5]	[6]
Ld F5 0(R2)	[1]	[3, 3]	[5, 6]	[7]	[8]
Add.d F6 F1 F2	[1]	[2, 4]	[]	[5]	[8]
Mult.d F7 F2 F3	[1]	[2, 6]	[]	[7]	[8]
Add R5 R3 R4	[2]	[3, 3]	[]	[4]	[8]
Sub R6 R4 R3	[2]	[4, 4]	[]	[5]	[9]
Sub.d F8 F3 F1	[2]	[3, 5]	[]	[6]	[9]
Mult.d F9 F1 F1	[2]	[3, 7]	[]	[8]	[9]
Addi R7 R3 100	[3]	[5, 5]	[]	[6]	[9]
Add.d F10 F4 F5	[3]	[8, 10]	[]	[11]	[12]
Mult.d F11 F6 F7	[3]	[8, 12]	[]	[13]	[14]
Sd F10 8(R2)	[3]	[4, 4]	[15, 16]	[]	[15]

The 12 instructions are issued in 3 cycles instead of 12, and the program takes 16 cycles instead of 21. The two loads issued in cycle 1 use the address adder in cycles 2 and 3, and `Add R5` and `Sub R6` issued in cycle 2 use the integer adder in cycles 3 and 4. Three results are broadcast in cycle 5 (`Ld F4`, `Add.d F6` and `Sub R6`). Four instructions commit in cycle 8 and four in cycle 9. What limits the program now is the dependencies: `Add.d F10` waits for the second load (WB in cycle 7), `Mult.d F11` waits for `Mult.d F7`, and the store writes memory when it commits, after `Mult.d F11`.

Final values (the same for all widths): R5=12, R6=2, R7=105, F4=10.0, F5=20.0, F6=4.0, F7=10.0, F8=2.5, F9=2.25, F10=30.0, F11=40.0, Mem[24]=30.0.

3.3 `test1_loop` (provided test case), issue width 2

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[2]	[3, 3]	[8, 12]	[13]	[25]
Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[3]	[4, 4]	[31, 35]	[]	[31]
Addi R1 R1 -4	[3]	[4, 4]	[]	[5]	[32]
Addi R2 R2 -4	[4]	[5, 5]	[]	[6]	[32]
Bne R1 R0 -12	[4]	[6, 6]	[]	[]	[33]
Addi R1 R1 -4	[7]	[8, 8]	[]	[9]	[33]
Addi R2 R2 -4	[7]	[9, 9]	[]	[10]	[34]
Bne R1 R0 -12	[8]	[10, 10]	[]	[]	[34]
Addi R1 R1 -4	[11]	[12, 12]	[]	[13]	[35]
Addi R2 R2 -4	[11]	[13, 13]	[]	[14]	[35]
Bne R1 R0 -12	[12]	[14, 14]	[]	[]	[36]
Add.d F20 F2 F2	[15]	[16, 19]	[]	[20]	[36]

Two instructions are issued per cycle. After each `Bne` nothing is issued until the cycle after its EX (issue 4, EX 6, next issue 7). The store starts writing memory in its commit cycle (31), after `Add.d F6` commits in cycle 30.

Final values (all widths): R1=0, R2=20, F2=1.0, F4=3.0, F6=7.0, F20=2.0, Mem[32]=7.0 (the other memory values do not change).

3.4 `test3_forwarding`, issue width 2

	ISSUE	EXE	MEM	WB	COMMIT
Add.d F1 F2 F3	[1]	[2, 4]	[]	[5]	[6]
Ld F4 8(R1)	[1]	[2, 2]	[3, 6]	[7]	[8]
Mult.d F5 F4 F2	[2]	[8, 27]	[]	[28]	[29]
Sd F5 0(R2)	[2]	[3, 3]	[30, 33]	[]	[30]

Ld F6 0(R2)	[3]	[4, 4]	[29]	[30]	[31]
Sd F1 4(R2)	[3]	[5, 5]	[34, 37]	[]	[34]
Ld F7 -8(R2)	[30]	[31, 31]	[38, 41]	[42]	[43]
Ld F8 4(R2)	[30]	[32, 32]	[33]	[34]	[43]
Add.d F9 F6 F8	[31]	[35, 37]	[]	[38]	[44]
Sd F9 2(R1)	[34]	[35, 35]	[45, 48]	[]	[45]
Ld F10 2(R1)	[34]	[36, 36]	[39]	[40]	[46]

- `Ld F6 0(R2)` gets its value from `Sd F5 0(R2)` by forwarding: one MEM cycle (29), the cycle after the store data is broadcast (28). `Ld F8` and `Ld F10` are also forwarded.
- `Ld F7 -8(R2)` has a different address from the stores before it, so it reads memory (4 cycles).
- `Ld F7` is issued in cycle 30 because the load/store queue (3 entries) is full until the first store commits.

Final values (all widths): F1=30.1, F4=7.5, F5=225.75, F6=225.75, F7=3.4, F8=30.1, F9=255.85, F10=255.85, Mem[12]=255.85, Mem[20]=225.75, Mem[24]=30.1.

3.5 `test6_class_example`

This is the example worked in the class slides (OOO example). Configuration: integer adder 1 RS, 1 cycle; FP adder 2 RS, 4 cycles; FP multiplier 2 RS, 10 cycles; load/store 2 entries, 1 cycle EX, 6 cycles MEM. R1=100, R2=200, F4=2.5, Mem[124]=10.0, Mem[232]=0.5.

Issue width 1:

	ISSUE	EXE	MEM	WB	COMMIT
Ld F6 24(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Ld F2 32(R2)	[2]	[3, 3]	[9, 14]	[15]	[16]
Mult.d F1 F2 F4	[3]	[16, 25]	[]	[26]	[27]
Mult.d F8 F6 F1	[4]	[27, 36]	[]	[37]	[38]
Add.d F10 F1 F2	[5]	[27, 30]	[]	[31]	[39]
Addi R2 R2 4	[6]	[7, 7]	[]	[8]	[40]

Issue width 2:

	ISSUE	EXE	MEM	WB	COMMIT
Ld F6 24(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Ld F2 32(R2)	[1]	[3, 3]	[9, 14]	[15]	[16]
Mult.d F1 F2 F4	[2]	[16, 25]	[]	[26]	[27]
Mult.d F8 F6 F1	[2]	[27, 36]	[]	[37]	[38]
Add.d F10 F1 F2	[3]	[27, 30]	[]	[31]	[38]
Addi R2 R2 4	[3]	[4, 4]	[]	[5]	[39]

All the cycles are the same as in the tables of the slides: 40 cycles with one issue per cycle and 39 with two. With width 2, `Mult.d F8` and `Add.d F10` commit in the same cycle (38). `run_tests.py` compares the two tables automatically.

The slides assume unpipelined FP units and the simulator has pipelined FP units, as the assignment says. In this program the result is the same, because the second multiply needs the result of the first one.

Final values (all widths): R2=204, F1=1.25, F2=0.5, F6=10.0, F8=12.5, F10=1.75.

4. How to run

```
cd code
python main.py                                  # test_case.txt
python main.py ../tests/test2_independent.txt 4   # input file, issue width
python run_tests.py                             # all tests, width 1 to 4
```

Python 3 is required; no other package is needed.